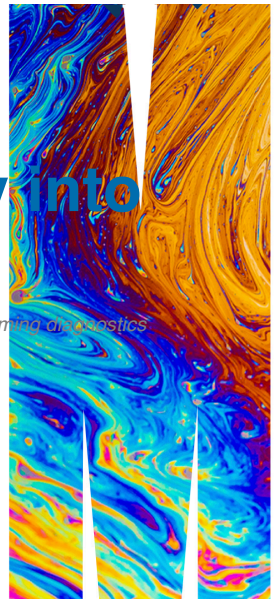
MONASH University

ETC5521: Diving Deeply into Data Exploration

Sculpting data using models, checking assumptions, co-dependency and performing diagnostics

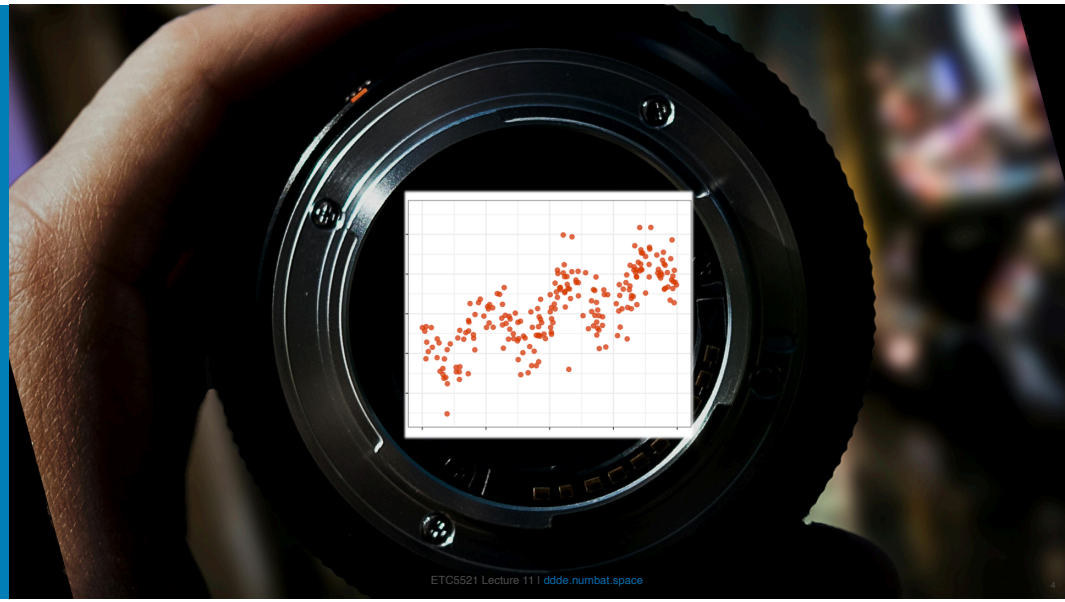
Professor Di Cook
Department of Econometrics and Business Statistics

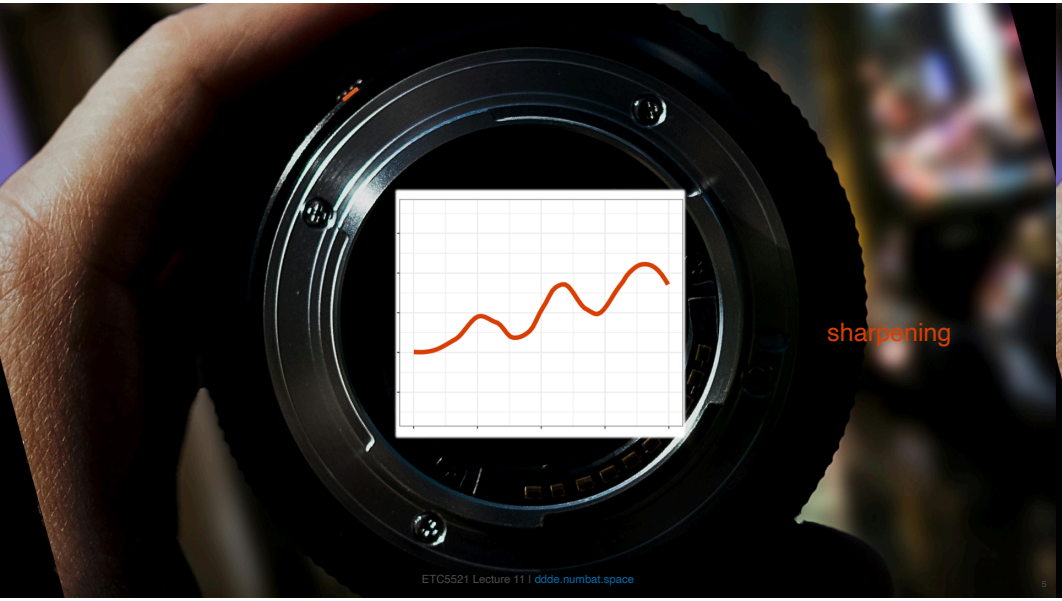


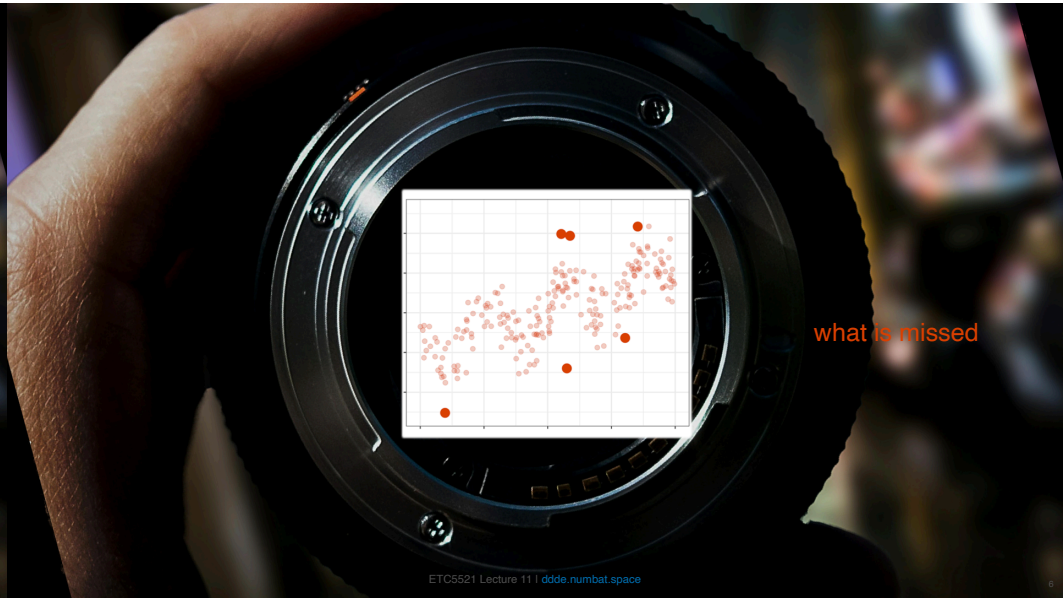
Outline

- Different types of model fitting
- Decomposing data from model
  - fitted
  - residual
- Diagnostic calculations
  - anomalies
  - leverage
  - influence

Models can be used to re-focus the view of data







Different types of model fitting

The basic form for fitting a model with data (response Y and predictors X) is:

$$Y = f(X) + \varepsilon$$

and X could be include multiple variables, $X = (X_1, X_2, \dots, X_p)$ where p is the number of variables. We have a sample of n observations,
$y_i, x_{i1}, \dots x_{ip}, \quad i = 1, \dots, n.$

- In a parametric model, the form of f is specified, e.g. $\beta_0 + \beta_1 X_1 + \beta_2 X_2 + \beta_3 X_1 X_2$, and one would estimate the parameters $\beta_0, \beta_1, \beta_2, \beta_3$.
  - Frequentist fitting assumes that parameters are fixed values.
  - In a Bayesian framework, the parameters are assumed to have a distribution, e.g. Gaussian.
- In a non-parametric model, the form of f is NOT specified but fitted from the data. May not have a specific functional form, and needs more data, typically. Imposes less assumptions. Can be done in a Bayesian framework.
- Different types of variables can change the model specification, e.g. binary or categorical Y, or temporal or spatial context.
- Different model products, e.g. fitted values or residuals, after the fit change the lens with which we view the data.

Parametric regression

Specification

Specify the

- functional form, e.g. function form is has linear and quadratic terms

$$f(X) = \beta_0 + \beta_1 X + \beta_2 X^2$$

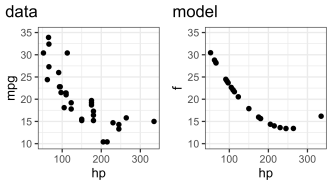
- distribution of errors, e.g.

$$\varepsilon \sim N(0, \sigma^2)$$

Fitting results in:

- fitted values, $\hat{y}$ (sharpening)
- residuals, $e = y - \hat{y}$ (what did we miss)

▸ Code



▸ Code

```
# A tibble: 3 × 5
  term         estimate std.error statistic  p.value
  <chr>           <dbl>     <dbl>     <dbl>    <dbl>
1 (Intercept)      20.1     0.544     36.9  6.15e-26
2 poly(hp, 2)1    -26.0     3.08      -8.46 2.51e- 9
3 poly(hp, 2)2     13.2     3.08       4.27 1.89e- 4
```

▸ Code

```
# A tibble: 1 × 12
  r.squared adj.r.squared sigma statistic      p.value    df
      <dbl>         <dbl> <dbl>     <dbl>        <dbl> <dbl>
1     0.756         0.739  3.08      45.0     1.30e-9     2
# ℹ 6 more variables: logLik <dbl>, AIC <dbl>, BIC <dbl>,
#   deviance <dbl>, df.residual <int>, nobs <int>
```

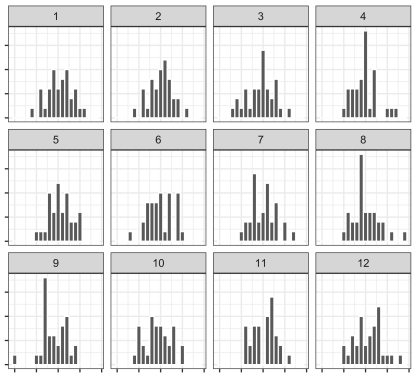

Diagnostics (1/3)

Residuals, $e = y - \hat{y}$ (*what doesn't the fitted model see?*)

- Should be consistent with a sample from the specified error model
- Should have no relationship with the response variable

Lineup | Normal? | Lineup | Relationship?

▸ Code



Diagnostics (2/3)

✏️ Activity: Hand-calculate the diagnostics

Your turn

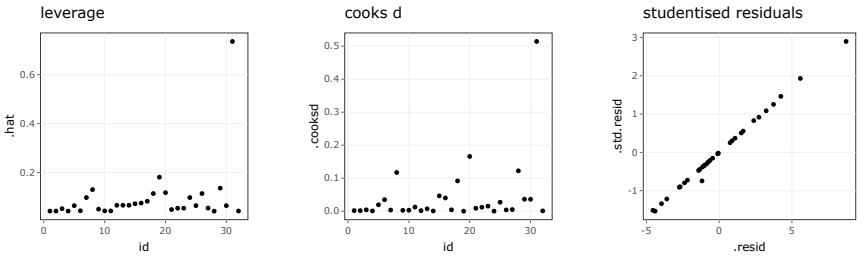
Use these 5 points from `mtcars` (`mpg` on `hp`), and the formulas from the previous slide, to hand-calculate the diagnostics for the simple linear model `mpg ~ hp`.

```
                  mpg  hp
Mazda RX4          21 110
Mazda RX4 Wag      21 110
Datsun 710         23  93
Hornet 4 Drive     21 110
Hornet Sportabout  19 175
```

1. Build the design matrix $\mathbf{X}$ (a column of 1's and the `hp` column), and compute the hat matrix $\mathbf{H} = \mathbf{X}(\mathbf{X}^\top\mathbf{X})^{-1}\mathbf{X}^\top$. Read off the leverages h_{ii} from the diagonal.
2. Fit the model and compute the residuals $e_i = y_i - \hat{y}_i$, and $\hat{\sigma} = \sqrt{\sum_i e_i^2/(n-p)}$ with $n = 5$, $p = 2$.
3. Compute the studentised residual e_i^* and Cook's distance D_i for each point.
4. Which point has the highest leverage? Does it also have the largest Cook's distance? Why or why not?
5. Check your answers against R's `hatvalues()`, `rstandard()` and `cooks.distance()`.

Diagnostics (3/3)

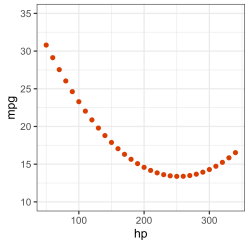
▸ Code



Simulation

Generate response values for un-collected predictor values

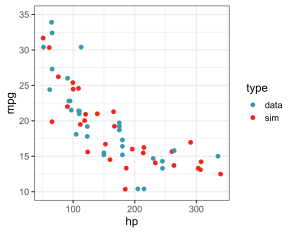
```
1 mt_full_fit <- tibble(hp = seq(50, 340, 10))
2 mt_full_fit <- mt_full_fit |>
3                mutate(mpg = predict(mtcars_fit, mt_ful
```



Simulate new samples

1 2 3

▸ Code



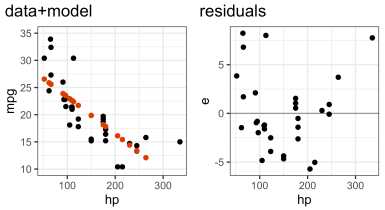
What can go wrong with parametric model?

Wrong specification

Specify function form is has only linear term

$$f(X) = \beta_0 + \beta_1 X$$

▸ Code



Polynomial

```
# A tibble: 3 × 5
  term         estimate std.error statistic  p.value
  <chr>           <dbl>     <dbl>     <dbl>    <dbl>
1 (Intercept)      20.1     0.544      36.9  6.15e-26
2 poly(hp, 2)1    -26.0     3.08       -8.46 2.51e- 9
3 poly(hp, 2)2     13.2     3.08        4.27 1.89e- 4

# A tibble: 1 × 12
  r.squared adj.r.squared sigma statistic      p.value    df
      <dbl>         <dbl> <dbl>     <dbl>        <dbl> <dbl>
1     0.756         0.739  3.08      45.0     1.30e-9      2
# ℹ 6 more variables: logLik <dbl>, AIC <dbl>, BIC <dbl>,
#   deviance <dbl>, df.residual <int>, nobs <int>
```

Linear

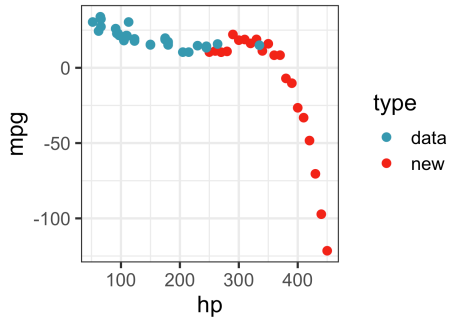
```
# A tibble: 2 × 5
  term        estimate std.error statistic  p.value
  <chr>          <dbl>     <dbl>     <dbl>    <dbl>
1 (Intercept)  30.1       1.63       18.4  6.64e-18
2 hp           -0.0682    0.0101     -6.74 1.79e- 7

# A tibble: 1 × 12
  r.squared adj.r.squared sigma statistic     p.value    df
      <dbl>         <dbl> <dbl>     <dbl>       <dbl> <dbl>
1     0.602         0.589  3.86      45.5 0.000000179     1
# ℹ 6 more variables: logLik <dbl>, AIC <dbl>, BIC <dbl>,
#   deviance <dbl>, df.residual <int>, nobs <int>
```


Extrapolating

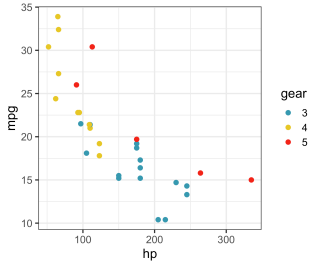
Generate response values for un-collected predictor values OUTSIDE of domain of collected data, can produce HALLUCINATIONS.

▸ Code

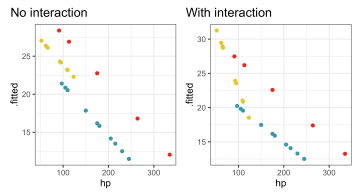


Multiple variables

Missing terms



▸ Code




```
# A tibble: 4 × 5
  term        estimate std.error statistic  p.value
  <chr>          <dbl>     <dbl>     <dbl>    <dbl>
1 (Intercept)  27.9       2.11       13.2  1.47e-13
2 hp           -0.0669    0.0110     -6.05 1.59e- 6
3 gear4         2.63      1.55        1.70 1.01e- 1
4 gear5         6.57      1.64        4.00 4.17e- 4

# A tibble: 6 × 5
  term        estimate std.error statistic  p.value
  <chr>          <dbl>     <dbl>     <dbl>    <dbl>
1 (Intercept) 25.3       2.65       9.55  5.51e-10
2 hp          -0.0522    0.0146    -3.59  1.36e- 3
3 gear4       15.3       3.86       3.95  5.31e- 4
4 gear5        7.47      3.81       1.96  6.05e- 2
5 hp:gear4    -0.127     0.0336    -3.78  8.25e- 4
6 hp:gear5    -0.00603   0.0193    -0.313 7.57e- 1
```

✏️ Activity: Polynomial interaction

Your turn

1. Fit two polynomial models (degree 2) of `mpg` on `hp`, one without and one with an interaction with `am`: `lm(mpg~poly(hp, 2)+am, data=mtcars)` and `lm(mpg~poly(hp, 2)*am, data=mtcars)`.
2. Plot the fitted values from each model against `hp`, coloured by `am`, as done for the linear model on the previous slide.
3. Compare the coefficient tables for the two models. How does allowing an interaction change the shape of the fitted curves, compared to the linear case? Is the interaction term necessary or useful for this data?

```
mtcars_fit_poly1 <- lm(mpg~poly(hp, 2)+am, data=mtcars)
mtcars_all_poly1 <- augment(mtcars_fit_poly1, mtcars)
tidy(mtcars_fit_poly1)

mtcars_fit_poly2 <- lm(mpg~poly(hp, 2)*am, data=mtcars)
mtcars_all_poly2 <- augment(mtcars_fit_poly2, mtcars)
tidy(mtcars_fit_poly2)

ggplot(mtcars_all_poly1, aes(x=hp, y=.fitted, colour=am)) +
  geom_point() +
  ggtitle("No interaction")

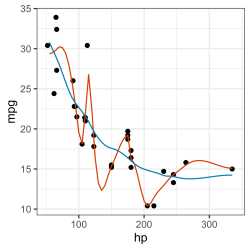
ggplot(mtcars_all_poly2, aes(x=hp, y=.fitted, colour=am)) +
  geom_point() +
  ggtitle("With interaction")
```


Non-parametric model

Smoothing splines

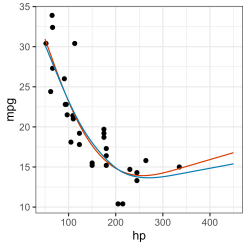
We've seen `loess`, which fits a linear model in a sliding window over predictor, where `span` controls size of window.

▸ Code



Smoothing splines, provide more advanced technique, and stability.

▸ Code



And are used to fit non-linear models to multiple predictors.

Logistic regression

- Not all parametric models assume normally distributed errors nor continuous responses.
- Logistic regression models the relationship between a set of explanatory variables $(x_{i1}, \ldots, x_{ik})$ and a set of **binary outcomes** Y_i for $i = 1, \ldots, n$.
- We assume that $Y_i \sim B(1, p_i) \equiv Bernoulli(p_i)$ and the model is given by

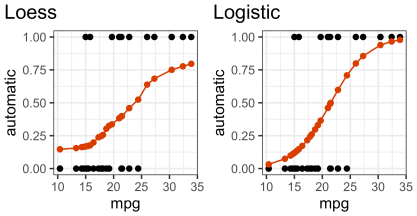
[Math Processing Error]

- Taking the exponential of both sides and rearranging we get

$$p_i = \frac{1}{1 + e^{-(\beta_0 + \beta_1 x_{i1} + \ldots + \beta_k x_{ik})}}.$$

- The function *[Math Processing Error]* is called the **logit** function, continuous with range $(-\infty, \infty)$, and if p is the probablity of an event, $f(p)$ is the log of the odds.

▸ Code



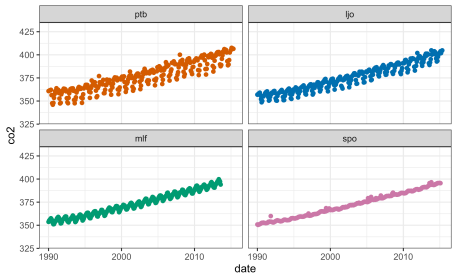
Slide a window and compute average (proportion) using loess, vs logistic function.

Time series

Trend and seasonality

▸ Code

Data | Trend | Seasonality



Data from Global CO2 monitoring stations

Exploring lags

Melbourne's temperature, from high to low!

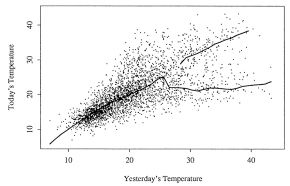


Figure 1. A Lagged Scatterplot of Each Day's Temperature Against the Previous Day's Temperature. Note the two "arms" on the right of the plot. The lines shown are from a modal regression discussed in Section 5.3.

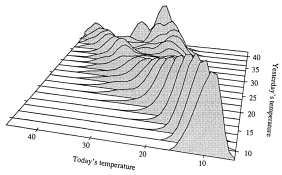


Figure 2. Stacked Conditional Density Plot of Temperature Conditional on the Previous Day's Temperature. The bimodality of the distribution of temperature following a hot day is more clear here than in Figure 1.

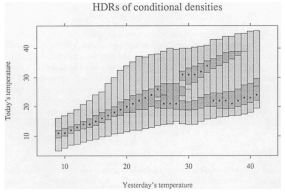


Figure 4. Highest Density Regions (50% and 99%) for Maximum Daily Temperature Conditional on the Previous Day's Maximum Temperature. Conditional modes are also marked (by ●) for each x value. Compare this plot with the scatterplot of Figure 1 and the modal regression plot of Figure 3.

Today plotted vertically, yesterday plotted horizontally. Different types of plots are different models applied to the data (lags).

Hyndman, Bashtannyk, Grunwald (1996)

High-dimensions

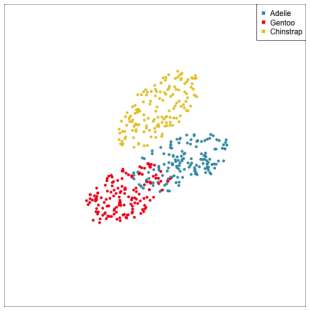
Groups

Data



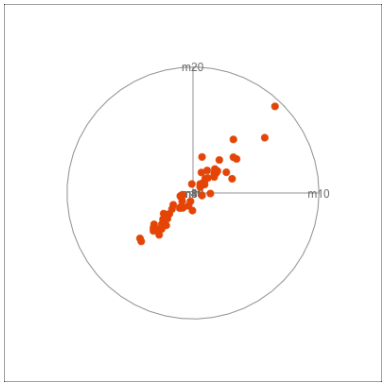
A little fuzzy.

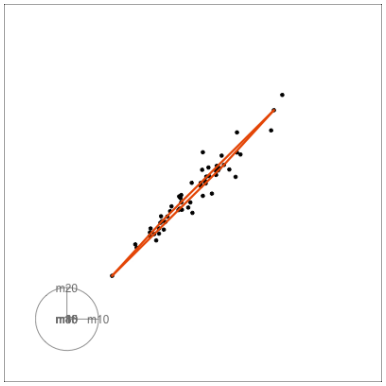
Model view



Clearer view. Misses some quirks.

Relationships





✏️ Activity: Reshape ratings to long form

Your turn

The `french_fries` data (`../data/french_fries.rda`) records sensory ratings of french fries over a 10-week experiment, on five scales: `potato`, `buttery`, `grassy`, `rancid`, `painty`.

1. Load the data with `load("../data/french_fries.rda")` and inspect it with `glimpse()`.
2. Convert it to long form with `pivot_longer()`, creating a `type` column (from the scale names) and a `rating` column (from the scale values).
3. Convert `time` to numeric, so weeks can be treated as a continuous predictor.

```
library(tidyr)
load("../data/french_fries.rda")
glimpse(french_fries)

ff_long <- french_fries |>
  pivot_longer(potato:painty, names_to = "type", values_to = "rating") |>
  mutate(time = as.numeric(time))
head(ff_long)
```


✏️ Activity: Raw view of ratings by week

Your turn

1. Using `ff_long`, plot `rating` against `time`, coloured by `type`, with `geom_point()` and `geom_smooth()`, faceted by `type`.
2. How easy is it to tell, from this raw/noisy view, whether ratings change over the 10 weeks for each scale?

```
ggplot(ff_long, aes(x = time, y = rating, colour = type)) +
  geom_point(size = 0.75) +
  geom_smooth() +
  facet_wrap(~type, ncol = 5) +
  theme(legend.position = "none")
```

✏️ Activity: Sharpen ratings with a model

Your turn

1. Fit a linear model of `rating` on `time` and `type`, allowing a different trend for each scale: `lm(rating ~ type*time - 1, data = ff_long)`.
2. Extract the fitted values (e.g. with `augment()`), and plot the fitted rating against `time`, coloured by `type`.
3. Compare this model view to the raw view on the previous slide – what has been sharpened? What variation has been hidden in the process?

```
library(broom)
ff_fit <- lm(rating ~ type*time - 1, data = ff_long)
ff_fitted <- augment(ff_fit)

ggplot(ff_fitted, aes(x = time, y = .fitted, colour = type)) +
  geom_line(linewidth = 1)
```

The model trades the noise of individual ratings for a clear picture of how each scale trends over the 10 weeks – a sharpened view, at the cost of hiding rater-to-rater and replicate variability.

Take-aways

- Models provide different lenses for extracting the patterns in the data
  - Sharpen
  - Exaggerate
  - Hallucinate
- Form a decomposition of the observed values into different strata
- Provide a multitude of other numerical quantities with which to see various aspects of the data.
- We are already using models, all the time, when making plots.

Resources

- Cook & Weisberg (1994) An Introduction to Regression Graphics
- Belsley, Kuh and Welsch (1980). Regression Diagnostics
- Hyndman, Bashtannyk, Grunwald (1996) Estimating and Visualizing Conditional Densities